0139. Word Break

Medium · 1-D Dynamic Programming
leetcode.com/problems/word-break

Problem

- Can s be segmented into a sequence of dictionary words (reuse allowed)?

Intuition

- Dp[i] = true if the prefix s[0..i) can be segmented. It is true if some j < i has dp[j] true and s[j..i) is a word.

Brute Force

- Recursion trying every split: exponential without memo.

Optimal Approach

- Dp[0] = true; dict as a set; maxLen = longest word.
- For i = 1..n, for j from i-1 down to max(0, i - maxLen): if dp[j] and dict has s[j..i), dp[i] = true; break.

Worked Example

```
s = "leetcode", dict [leet, code].

dp[0]=T
dp[4]: s[0..4)="leet" in dict, dp[0]=T -> T
dp[8]: s[4..8)="code" in dict, dp[4]=T -> T
answer true
```

Java Solution

```java
public class WordBreak {
    public boolean wordBreak(String s, List<String> wordDict) {
        Set<String> dict = new HashSet<>(wordDict);
        int maxLen = 0;
        for (String w : wordDict) maxLen = Math.max(maxLen, w.length());
        boolean[] dp = new boolean[s.length() + 1];
        dp[0] = true;
        for (int i = 1; i <= s.length(); i++)
            for (int j = i - 1; j >= Math.max(0, i - maxLen); j--)
                if (dp[j] && dict.contains(s.substring(j, i))) { dp[i] = true; break; }
        return dp[s.length()];
    }
}
```

Complexity

- Time: O(n maxLen maxLen) with substring hashing. Space: O(n).

Pitfalls

- Bounding j by the longest word length avoids pointless substring checks.